# HPL 与 HPCG 性能优化报告

姓名：林佳逸 | 年级专业：24 级软件工程 | 学号：202440025315

## 一、环境准备：

### （1）硬件：

先来介绍一下我的硬件设备，这次任务用到的是我个人的联想 R7000 笔记本，16GB 内存，搭载的 CPU 是 AMD Ryzen 7 7840H 处理器，配备 8 个物理核心、16 个逻辑线程以及三级缓存，默频 3.8GHz，在本任务中我没有选择拉高功耗超频 CPU 来换取更高的性能。经过预先了解，我知道 HPL 和 HPCG 两个任务都是考核 CPU 的计算能力，未对 GPU 有要求，所以我先在电脑的控制界面上做出如下配置，为稍后的任务准备较好的物理条件：

上面的配置简单来说就是放宽 CPU 的功耗墙和温度墙，以及避免动态分配机制为了平衡整机功耗将 CPU 的功耗配额分配给 GPU，但是我没有强制给 cpu 拉高功耗进行超频，并且这些配置优化前后保持

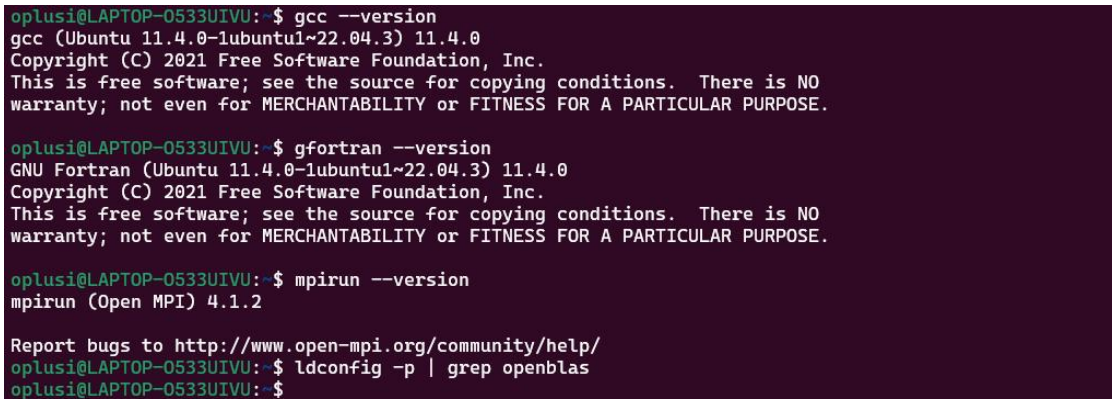
一致，不算在优化过程中，仅仅是为了提供较好的物理条件。

（2）软件：

本任务采用的软件环境是本机上安装运行的 WSL2，系统版本为 Ubuntu 22.04.5。选择 WSL2 的原因在于，一是他更加接近原生 Linux，更加贴合比赛环境，二是他相比虚拟机能够更好的发挥硬件性能，WSL2 可以直接调度物理硬件不用通过虚拟机虚拟化层的转化，从而带来性能的提升。优化前后的任务都运行在 WSL2 上，都保持一致的软件环境。

## 二、HPL 和 HPCG 部署跑通

### （1）HPL 的部署




首先我根据老师给予的赛事文档指导，了解到 HPL2.3 的编译依赖四个组件，我先检查一下本地环境中四个依赖的状况，如上图可见，gcc、Fortran、MPI 三类组件已经安装，维度缺少 BLAS 库。

```
oplusi@LAPTOP-O533UIVU:~$ sudo apt update
sudo apt install -y libopenblas-dev
[sudo] password for oplusi:
Hit:1 https://mirrors.ustc.edu.cn/ubuntu jammy InRelease
Get:2 https://mirrors.ustc.edu.cn/ubuntu jammy-updates InRelease [128 kB]
Hit:3 https://mirrors.ustc.edu.cn/ubuntu jammy-backports InRelease
Hit:4 https://mirrors.ustc.edu.cn/ubuntu jammy-security InRelease
Get:5 https://mirrors.ustc.edu.cn/ros2/ubuntu jammy InRelease [4682 B]
Get:6 https://mirrors.ustc.edu.cn/ubuntu jammy-updates/main amd64 Packages [3909 kB]
Get:7 https://mirrors.ustc.edu.cn/ubuntu jammy-updates/universe amd64 Packages [1289 kB]
```

```
oplusi@LAPTOP-O533UIVU:~$ ldconfig -p | grep -i openblas
        libopenblas.so.0 (libc6,x86-64) => /lib/x86_64-linux-gnu/libopenblas.so.0
        libopenblas.so (libc6,x86-64) => /lib/x86_64-linux-gnu/libopenblas.so
oplusi@LAPTOP-O533UIVU:~$
```

如上图所示，直接安装然后检查是否生成动态链接库就好了，有了就是安装到位了，下面开始拉取 HPL2.3 开始编译。

```
oplusi@LAPTOP-O533UIVU:~$ mkdir -p ~/asc27/src
oplusi@LAPTOP-O533UIVU:~$ cd ~/asc27/src
oplusi@LAPTOP-O533UIVU:~/asc27/src$ wget -c https://www.netlib.org/benchmark/hpl/hpl-2.3.tar.gz
--2026-10-25 13:46:23--  https://www.netlib.org/benchmark/hpl/hpl-2.3.tar.gz
Resolving www.netlib.org (www.netlib.org)... 160.36.239.231
Connecting to www.netlib.org (www.netlib.org)|160.36.239.231|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://netlib.org/benchmark/hpl/hpl-2.3.tar.gz [following]
--2026-10-25 13:46:24--  https://netlib.org/benchmark/hpl/hpl-2.3.tar.gz
Resolving netlib.org (netlib.org)... 160.36.239.231
Connecting to netlib.org (netlib.org)|160.36.239.231|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 660871 (645K) [application/x-gzip]
Saving to: ‘hpl-2.3.tar.gz’

hpl-2.3.tar.gz             100%[=====================================>] 645.38K  16.3KB/s    in 36s

2026-10-25 13:47:05 (18.0 KB/s) - ‘hpl-2.3.tar.gz’ saved [660871/660871]

oplusi@LAPTOP-O533UIVU:~/asc27/src$
```

这一步非常简单，创建个工作空间，拉取 hpl 安装包然后解压即可。

```
oplusi@LAPTOP-O533UIVU:~/asc27$ git clone https://github.com/fanchenyou/scnu_asc.git
Cloning into 'scnu_asc'...
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 12 (delta 2), reused 11 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (12/12), 5.43 KiB | 2.71 MiB/s, done.
Resolving deltas: 100% (2/2), done.
oplusi@LAPTOP-O533UIVU:~/asc27$ ls
scnu_asc  src
```

然后再准备好老师提供的测试数据，git 拉取即可。

```
oplusi@LAPTOP-O533UIVU:~/asc27$ git clone https://github.com/fanchenyou/scnu_asc.git
Cloning into 'scnu_asc'...
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 12 (delta 2), reused 11 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (12/12), 5.43 KiB | 2.71 MiB/s, done.
Resolving deltas: 100% (2/2), done.
oplusi@LAPTOP-O533UIVU:~/asc27$ ls
scnu_asc  src
oplusi@LAPTOP-O533UIVU:~/asc27$ cd src/
oplusi@LAPTOP-O533UIVU:~/asc27/src$ cd hpl-2.3/
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpl-2.3$ ls
AUTHORS     ChangeLog  Makefile     README  acinclude.m4  config.sub    include     missing  www
BUGS        HISTORY    Makefile.am  THANKS  aclocal.m4    configure     install-sh  setup
COPYING     INSTALL    Makefile.in  TODO    compile       configure.ac  makes       src
COPYRIGHT   Make.top   NEWS         TUNING  config.guess  depcomp       man         testing
```

这里是进到工作目录里面去，顺带看看目录里面有什么。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpl-2.3$ cp ~/asc27/scnu_asc/Make.hpl_linux Make.linux
```

将老师提供的测试数据中的编译配置 copy 一份过来，这里命名要遵循 HPL 的习惯，命名为 Make.linux 方便后继编译。接下来就是用 nano 编辑这一份配置文件使得其适配当前的环境

```
# ----------------------------------------------------------------------
# - HPL Directory Structure / HPL library ------------------------------
# ----------------------------------------------------------------------
#
TOPdir       = /home/oplusi/asc27/src/hpl-2.3
INCdir       = $(TOPdir)/include
BINdir       = $(TOPdir)/bin/$(ARCH)
LIBdir       = $(TOPdir)/lib/$(ARCH)
#
HPLlib       = $(LIBdir)/libhpl.a
```

这里我就修改了 TOPdir 根目录，将其替换为实际的 hpl-2.3 所在路径，下面的不用修改，会根据顶部根目录的宏自己推算出来。

```
# ----------------------------------------------------------------------
# - Message Passing library (MPI) --------------------------------------
# ----------------------------------------------------------------------
# MPinc tells the  C  compiler where to find the Message Passing library
# header files,  MPlib  is defined  to be the name of  the library to be
# used. The variable MPdir is only used for defining MPinc and MPlib.
#
MPdir        =
MPinc        =
MPlib        =
#
```

本机上没有 MPICH，我们用的是 OpenMPI，为了防止编译器找错或者混用库文件，直接清空让编译器不再查找。

```
# ----------------------------------------------------------------------
# - Linear Algebra library (BLAS or VSIPL) -----------------------------
# ----------------------------------------------------------------------
# LAinc tells the  C  compiler where to find the Linear Algebra  library
# header files,  LAlib  is defined  to be the name of  the library to be
# used. The variable LAdir is only used for defining LAinc and LAlib.
#
LAdir        =
LAinc        =
LAlib        = -lopenblas -lm -lpthread
```

这里 LAdir 和 LAinc 清空不用显示配置就是因为，OpenBLAS 是通过 apt 安装的，就在系统默认的库搜索路径里，编译器自己能找到。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpl-2.3$ make arch=linux
make -f Make.top startup_dir      arch=linux
make[1]: Entering directory '/home/oplusi/asc27/src/hpl-2.3'
mkdir include/linux
mkdir lib
mkdir lib/linux
mkdir bin
mkdir bin/linux
make[1]: Leaving directory '/home/oplusi/asc27/src/hpl-2.3'
make -f Make.top startup_src      arch=linux
make[1]: Entering directory '/home/oplusi/asc27/src/hpl-2.3'
make -f Make.top leaf le=src/auxil        arch=linux
make[2]: Entering directory '/home/oplusi/asc27/src/hpl-2.3'
( cd src/auxil ; mkdir linux )
( cd src/auxil/linux ; \
            ln -s /home/oplusi/asc27/hpl-2.3/Make.linux Make.inc )
make[2]: Leaving directory '/home/oplusi/asc27/src/hpl-2.3'
make -f Make.top leaf le=src/blas         arch=linux
make[2]: Entering directory '/home/oplusi/asc27/src/hpl-2.3'
( cd src/blas ; mkdir linux )
( cd src/blas/linux ; \
            ln -s /home/oplusi/asc27/hpl-2.3/Make.linux Make.inc )
```

然后就是编译了，make arch=linux 开始编译，之前的命名习惯就是为了 make arch=xxx 的时候，编译器会去找 Make.xxx 的编译配置。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpl-2.3$ ls -lh bin/linux/xhpl
-rwxr-xr-x 1 oplusi oplusi 205K Oct 25 16:28 bin/linux/xhpl
```

编译成功后检查一下，出现 xhpl 就是编译成功了，可以开始跑测试了。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpl-2.3$ cp ~/asc27/scnu_asc/HPL.dat ~/asc27/src/hpl-2.3/bin/linu
x/
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpl-2.3$ cd bin/linux/
```

跑测试要先移步到子目录里的 bin/linux 里面，然后记得将老师提

供的测试配置也复制一份过来，一会儿测试时将会参照这一份文件。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpl-2.3/bin/linux$ mpirun -np 1 ./xhpl
================================================================================
HPLinpack 2.3  --  High-Performance Linpack benchmark  --   December 2, 2018
Written by A. Petitet and R. Clint Whaley,  Innovative Computing Laboratory, UTK
Modified by Piotr Luszczek, Innovative Computing Laboratory, UTK
Modified by Julien Langou, University of Colorado Denver
================================================================================
```

输入命令即可开始测试，需要注意的是，老师给出的配置中 P*Q=1，所以进程数量要匹配，-np 就得等于 1。

```
An explanation of the input/output parameters follows:
T/V    : Wall time / encoded variant.
N      : The order of the coefficient matrix A.
NB     : The partitioning blocking factor.
P      : The number of process rows.
Q      : The number of process columns.
Time   : Time in seconds to solve the linear system.
Gflops : Rate of execution for solving the linear system.

The following parameter values will be used:

N      :    5040
NB     :     128
PMAP   : Row-major process mapping
P      :       1
Q      :       1
PFACT  :   Right
NBMIN  :       4
NDIV   :       2
RFACT  :   Crout
BCAST  :  1ringM
DEPTH  :       1
SWAP   : Mix (threshold = 64)
L1     : transposed form
U      : transposed form
EQUIL  : yes
ALIGN  : 8 double precision words

--------------------------------------------------------------------------------

- The matrix A is randomly generated for each test.
- The following scaled residual check will be computed:
      ||Ax-b||_oo / ( eps * ( || x ||_oo * || A ||_oo + || b ||_oo ) * N )
- The relative machine precision (eps) is taken to be               1.110223e-16
- Computational tests pass if scaled residuals are less than                16.0
```

运行时，可以看上面信息确认配置，这些配置我稍后在优化章节再提及，可以看到下方有一个提示说缩放后的残差小于 16，那么就算通过，我的理解是计算残差就是校验结果，16 以下就代表结果正确。

```
================================================================================
T/V                N    NB     P     Q               Time                 Gflops
--------------------------------------------------------------------------------
WR11C2R4        5040   128     1     1               1.39             6.1256e+01
```

这个便是一次测试的结果了，只代表跑通了，正式基线还需要 5 次测试取平均值避免软硬件偶然波动导致虚高虚低的情况。

HPL 就跑通了，接下来就是 HPCG 的安装测试。

（2）HPCG

```
oplusi@LAPTOP-O533UIVU:~/asc27/src$ command -v mpicxx
/usr/bin/mpicxx
oplusi@LAPTOP-O533UIVU:~/asc27/src$ mpicxx --version
g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

首先检查一下依赖，HPCG 源码主要依靠 c++编写，所以有 c++ 的编译器便准备就绪了。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpl-2.3/bin/linux$ cd ~/asc27/src
oplusi@LAPTOP-O533UIVU:~/asc27/src$ wget -c https://github.com/hpcg-benchmark/hpcg/archive/refs/tags
/HPCG-release-3-0-0.tar.gz
--2026-10-25 17:00:44--  https://github.com/hpcg-benchmark/hpcg/archive/refs/tags/HPCG-release-3-0-0
.tar.gz
Resolving github.com (github.com)... 20.205.243.166
```

```
oplusi@LAPTOP-O533UIVU:~/asc27/src$ tar -xzf HPCG-release-3-0-0.tar.gz
oplusi@LAPTOP-O533UIVU:~/asc27/src$ mv hpcg-HPCG-release-3-0-0 hpcg-3.0
oplusi@LAPTOP-O533UIVU:~/asc27/src$
```

和之前一样，拉去 HPCG 软件包，然后解压顺便为了以后方便敲命令和查看，给软件目录更换一下名字。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src$ cd ~/asc27/src/hpcg-3.0
```

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0$ ls
BUGS       HISTORY  LICENSE   Makefile.ext  README  TUNING  configure  setup  tools       web
COPYRIGHT  INSTALL  Makefile  QUICKSTART    TODO    bin     index.doc  src    unittesting
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0$ mkdir build_linux
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0$
```

进入到工作目录，看看都有些什么，然后再建立一个存放编译文件的工作区就可以了。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0$ cp ~/asc27/scnu_asc/Make.hpcg_linux setup/Make.scnu
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0$ nano setup/Make.scnu
```

Copy 一份老师提供的配置文件，和 HPL 一样，准备用 nano 修改。

这里 Make 文件的后缀比较随意，不用像 HPL 里面那样严格。

```
# ----------------------------------------------------------------------
# - HPCG Directory Structure / HPCG library ---------------------------
# ----------------------------------------------------------------------
#
TOPdir       = .
SRCdir       = $(TOPdir)/src
INCdir       = $(TOPdir)/src
BINdir       = $(TOPdir)/bin
```

这份配置文件，我只修改了 TOPdir，改为“.”，也就是当前目录，下面的配置会根据 TOPdir 的宏自己推算而出。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0/build_linux$ ../configure scnu
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0/build_linux$ make -j8
mpicxx -c -DHPCG_NO_OPENMP -I./src -I./src/scnu -I/usr/lib/x86_64-linux-gnu/openmpi/include -O3 -
rize -ftree-vectorizer-verbose=0 -I../src ../src/main.cpp -o src/main.o
mpicxx -c -DHPCG_NO_OPENMP -I./src -I./src/scnu -I/usr/lib/x86_64-linux-gnu/openmpi/include -O3 -
rize -ftree-vectorizer-verbose=0 -I../src ../src/CG.cpp -o src/CG.o
mpicxx -c -DHPCG_NO_OPENMP -I./src -I./src/scnu -I/usr/lib/x86_64-linux-gnu/openmpi/include -O3 -
rize -ftree-vectorizer-verbose=0 -I../src ../src/CG_ref.cpp -o src/CG_ref.o
```

然后转到之前创建的工作空间，开始加载配置，scnu 对应上之前 Make.scnu 的后缀，然后 make -j8 开始编译。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0/build_linux$ ls bin/xhpcg
bin/xhpcg
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0/build_linux$
```

编译后检查一下有无 xhpcg 出现，如果有的话便是编译成功，可以开始测试了。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0/build_linux$ cp ~/asc27/scnu_asc/hpcg.dat bin/hpcg.dat
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0/build_linux$ cd bin
```

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpcg-3.0/build_linux/bin$ mpirun -np 1 ./xhpcg \
--nx=16 --ny=16 --nz=16 --rt=30
```

复制一份老师提供的测试配置过来，然后切换到子目录 bin 中去，输入命令开始测试。老师提供的测试配置中就是--nx=16 --ny=16 --nz=16 --rt=30，这里显式输入是因为不带这些参数的话，HPCG 会自动启用 QuickPath，赛题说不允许使用 QuickPath，所以这里显示输入参数规避 QuickPath。

```
---------- Final Summary ----------:
HPCG result is VALID with a GFLOP/s rating of: 3.65435
    HPCG 2.4 Rating (for historical value) is: 3.75525
Reference version of ComputeDotProduct used: Performance results are most likely suboptimal
Reference version of ComputeSPMV used: Performance results are most likely suboptimal
Reference version of ComputeMG used: Performance results are most likely suboptimal
Reference version of ComputeWAXPBY used: Performance results are most likely suboptimal
Results are valid but execution time (sec) is: 31.8566
     Official results execution time (sec) must be at least: 1800
```

这里就是最后的总结，值得关注的指标就是 GFLOP/s rating，这是程序通过所有正确性校验后的基线性能，后继优化都要对比这个，也是要重复测算多次取平均值来避免偶然性的，下面四个提示说我们现在使用的都是原始参考，后继或许我们可以从这里入手去优化性能。

现在 HPL 和 HPCG 的安装测试都跑通了，接下来就是调优。

## 三、HPL 和 HPCG 调优

### （1）HPL 和 HPCG 概述

优化之前先说一说我对 HPL 和 HPCG 的理解，来看看可以优化的着力点。

首先是 HPL，我理解 HPL 就是让 CPU 求解一个线性方程组 Ax=b，求的是未知向量 x，计算机求解的方式不是求出 A 的逆然后再用 A 的逆乘上 b，而是使用了 LU 分解，在这个过程中，计算机会将这个大矩阵拆成 NB*NB 大小的小矩阵去进行计算，期间会使用大量的矩阵乘法。所以 HPL 从内存中读取一次数据就可以计算很多次，对内存的访问性能要求不高，HPL 的优化关键点可能在于调整像 NB 这样的参数。

然后是 HPCG，HPCG 也是让 CPU 求解一个线性方程组，不同之处在于这个线性方程组 Ax=b 中，这个 A 是非常稀疏的，一行里面就只有少数的非零元素，这就使得 HPCG 的计算强度不高，但是由于

在求解过程中需要读取分布不规则的非零元素，所以对内存的访问性能有比较高的要求，结合之前看到的提示信息，或许我们可以通过优化核心函数，改变内存访问的行为方式来进行性能的优化。

（2）HPL 的优化

```
WR11C2R4       5040   128     1     1               1.39             6.1628e+01
WR11C2R4       5040   128     1     1               1.39             6.1572e+01
WR11C2R4       5040   128     1     1               1.36             6.2807e+01
WR11C2R4       5040   128     1     1               1.36             6.2692e+01
WR11C2R4       5040   128     1     1               1.36             6.2631e+01
```

首先来看五次基线性能，基线只使用了老师给出的测试配置的数据，可以看到 5 次平均 GFLOPS/s 都在 62 左右。

```
# ----------------------------------------------------------------------
# - Compilers / linkers - Optimization flags ---------------------------
# ----------------------------------------------------------------------
#
CC           = mpicc
CCNOOPT      = $(HPL_DEFS)
CCFLAGS      = $(HPL_DEFS) -O3 -march=native -funroll-loops
#
LINKER       = mpif77
LINKFLAGS    =
#
ARCHIVER     = ar
ARFLAGS      = r
RANLIB       = echo
#
# ----------------------------------------------------------------------
```

我对 HPL 的第一个优化是在编译选项上，这个是在我看 hpcg 的编译配置时候发现的，开启 O3 编译优化在算法竞赛中比较常见，有一些 TLE 的题目开启 O3 优化后反而能 AC 了。

```
================================================================================
T/V                N    NB     P     Q               Time                 Gflops
--------------------------------------------------------------------------------
WR11C2R4       5040   128     1     1               1.31             6.4968e+01
```

相比基线结果，编译优化带来了一定的提升，接下来跑五次看看优化稳不稳定。

```
WR11C2R4       5040   128     1     1              1.31             6.5170e+01
WR11C2R4       5040   128     1     1              1.31             6.5401e+01
WR11C2R4       5040   128     1     1              1.30             6.5573e+01
WR11C2R4       5040   128     1     1              1.30             6.5754e+01
WR11C2R4       5040   128     1     1              1.31             6.5197e+01
```

可以看到，编译带来的提升还是比较稳定的，相比基线的话大概是 2.5 GFLOPS/s 左右的提升。

```
oplusi@LAPTOP-O533UIVU:~/asc27/src/hpl-2.3/bin/linux$ nano HPL.dat
```

接下来就是修改配置文件调整参数来达到更好的性能了。

```
HPLinpack benchmark input file
Innovative Computing Laboratory, University of Tennessee
HPL.out      output file name (if any)
6            device out (6=stdout,7=stderr,file)
1            # of problems sizes (N)
5040         Ns
1            # of NBs
128          NBs
0            PMAP process mapping (0=Row-,1=Column-major)
1            # of process grids (P x Q)
1            Ps
1            Qs
16.0         threshold
1            # of panel fact
2            PFACTs (0=left, 1=Crout, 2=Right)
1            # of recursive stopping criterium
4            NBMINs (>= 1)
1            # of panels in recursion
2            NDIVs
1            # of recursive panel fact.
1            RFACTs (0=left, 1=Crout, 2=Right)
1            # of broadcast
1            BCASTs (0=1rg,1=1rM,2=2rg,3=2rM,4=Lng,5=LnM)
1            # of lookahead depth
1            DEPTHs (>=0)
2            SWAP (0=bin-exch,1=long,2=mix)
64           swapping threshold
0            L1 in (0=transposed,1=no-transposed) form
0            U  in (0=transposed,1=no-transposed) form
1            Equilibration (0=no,1=yes)
8            memory alignment in double (> 0)
```

进入 HPL.dat 的配置文件，可以看到一系列参数。通过网络资料的学习，一些 CSDN 的博文上指出，HPL 影响性能的调参主要是四个：Ns、NBs、Ps 和 Qs，分别就是问题规模，切块大小和每一个进

程的网格如何切分。下面我做了一个初步的尝试。

```
HPLinpack benchmark input file
Innovative Computing Laboratory, University of Tennessee
HPL.out      output file name (if any)
6            device out (6=stdout,7=stderr,file)
1            # of problems sizes (N)
5040         Ns
1            # of NBs
128          NBs
0            PMAP process mapping (0=Row-,1=Column-major)
1            # of process grids (P x Q)
2            Ps
4            Qs
16.0         threshold
1            # of panel fact
2            PFACTs (0=left, 1=Crout, 2=Right)
1            # of recursive stopping criterium
4            NBMINs (>= 1)
1            # of panels in recursion
2            NDIVs
1            # of recursive panel fact.
1            RFACTs (0=left, 1=Crout, 2=Right)
1            # of broadcast
1            BCASTs (0=1rg,1=1rM,2=2rg,3=2rM,4=Lng,5=LnM)
1            # of lookahead depth
1            DEPTHs (>=0)
2            SWAP (0=bin-exch,1=long,2=mix)
64           swapping threshold
0            L1 in (0=transposed,1=no-transposed) form
0            U  in (0=transposed,1=no-transposed) form
1            Equilibration (0=no,1=yes)
8            memory alignment in double (> 0)
```

我将 Ps 和 Qs 分别调为 2 和 4，让 MPI 进程以 2*4 的规格去切分矩阵，总进程数为 8，先做一步尝试。

```
WR11C2R4       5040   128     2     4              0.49             1.7484e+02
WR11C2R4       5040   128     2     4              0.46             1.8590e+02
WR11C2R4       5040   128     2     4              0.46             1.8547e+02
WR11C2R4       5040   128     2     4              0.47             1.8171e+02
WR11C2R4       5040   128     2     4              0.47             1.8305e+02
```

可以看到效果显著，带来了近乎 3 倍的性能提升，多进程发挥了关键作用，在单进程基线中，庞大的矩阵运算仅依靠单核执行，无法榨干 CPU 的浮点算力；而在 8 进程网格下，矩阵被切分并分配给 8

个物理核心并发处理，释放了系统的并行计算潜能。接下来就是扫描一下参数看看最佳配置如何。

### 3. HPL.dat 参数配置 (AMD 专属)

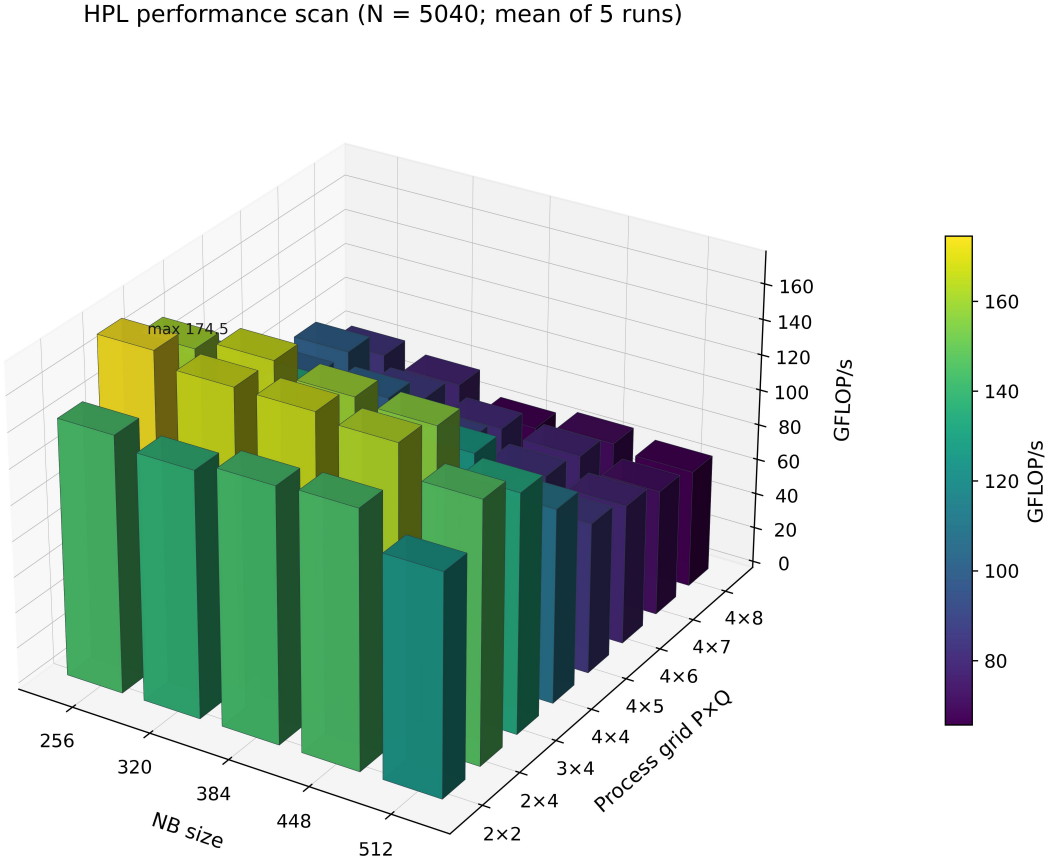
与通用配置类似，但部分参数针对 Zen 架构有特定推荐值。

| 参数 | 说明 | AMD Zen 3/4 推荐值 |
| --- | --- | --- |
| **N** (Problem Size) | 矩阵维度 | $N \approx \sqrt{\frac{\text{总内存} \times 0.9}{8}}$ |
| **NB** (Block Size) | 分块大小 | **384** (Genoa/Milan 架构的最佳甜点) |
| **P, Q** | 进程网格 | $P \times Q =$ 总插槽数 (Sockets)<br>建议每 Socket 运行 1 个 MPI 进程 |

**进程模型差异**

在 Intel 平台上，有时会尝试每 NUMA 节点一个 MPI 进程。 在 **AMD 平台** 上，最佳实践通常是 **每个物理 CPU 插槽 (Socket) 一个 MPI 进程**，并在该进程内开启与核心数相等的 OpenMP 线程。

进行扫描之前，我去网上查阅了一下资料，我的 CPU 属于 Zen 4 架构，对于 Zen 4 架构，网上的推荐值如上。对于问题规模，该 WSL 分配到 4GB 内存，5040 差不多就合适够用了，虽然会因为运行时间较短而有一些抖动。分块大小建议的甜点值为 384，P*Q 最好等于总插槽数量，由于我只有一块 CPU 所以物理插槽数目只有 1，但是有八个物理核心，所以不一定 P*Q 要等于 1，以上只是其他人的推荐参数，实际需要扫描后才能根据本机情况得出结论，所以我设置了一组扫描实验。




这是扫描结果，最高平均性能是 174.5 GFLOP/s。这个性能是在 P=2、Q=4、NB=256 的时候得到的，NB=384 附近的性能都大差不差偶有波动，影响较大的还是 P 与 Q 的参数设置，总之最后我们的优化结果便是从基准性能 62.186 GFLOP/s 到 174.5 GFLOP/s

### （3）HPCG 的优化

HPCG 的优化如前面所说，主要集中在核心函数的优化上，至于编译配置方面的优化我尝试过，但却是负优化，GFLOP/s 直接下降到 0.8 多，属于是错误优化，故不展示。

参数方面，经过查询资料，比较关键的是问题规模这里先做个演示：

```
HPCG result is VALID with a GFLOP/s rating of: 3.50882
Results are valid but execution time (sec) is: 30.855
HPCG result is VALID with a GFLOP/s rating of: 3.57588
Results are valid but execution time (sec) is: 31.1193
HPCG result is VALID with a GFLOP/s rating of: 3.51897
Results are valid but execution time (sec) is: 29.5125
HPCG result is VALID with a GFLOP/s rating of: 3.60715
Results are valid but execution time (sec) is: 30.237
HPCG result is VALID with a GFLOP/s rating of: 3.56571
Results are valid but execution time (sec) is: 30.4143
```

上面是保持老师给的默认参数下的五次基线平均测试水平，可以看到基本都稳定再 3.5~3.6 GFLOP/s 左右。此时的问题规模是，nx=16，ny=16，nz=16，属于较小的问题规模。

```
HPCG result is VALID with a GFLOP/s rating of: 1.78451
Results are valid but execution time (sec) is: 31.0777
HPCG result is VALID with a GFLOP/s rating of: 1.89208
Results are valid but execution time (sec) is: 31.8266
HPCG result is VALID with a GFLOP/s rating of: 1.81794
Results are valid but execution time (sec) is: 30.6765
HPCG result is VALID with a GFLOP/s rating of: 1.92419
Results are valid but execution time (sec) is: 31.3545
HPCG result is VALID with a GFLOP/s rating of: 1.73311
Results are valid but execution time (sec) is: 32.1761
```

接下来，只修改问题规模，把 nx=16、ny=16、nz=16 修改为 nx=64、ny=64、nz=64，然后再跑一遍，可以看到 GFLOP/s 腰斩。我查了查资料，这是因为当网格规模为 16、16、16 时，整个稀疏矩阵和向量数据体积很小，完全可以被全部塞进 CPU 的三级缓存中。此时 CPU 几乎不需要访问主存，计算单元不再等数据，因此跑出了极高的 GFLOP/s。然而，当规模扩大到 64、64、64 时，矩阵加上各种中间向量的数据量远超 L3 缓存容量，需要从主存中调取了。也就是说从口袋里找东西变成从仓库里面找东西了，速度自然下降了。

这个情况是正常的，虽然我是 AMD 的 CPU，但是我看到 intel 的网络文档中就是推荐把问题规模调整到 CPU 缓存装不下，但是设备

内存还能承受的地步，这样可以比较真实的反馈 CPU 实际性能，不会虚高。接下来的优化我都是基于 nx=64、ny=64、nz=64 的配置进行的。

HPCG 的优化主要涉及到这些函数，里面有一些是新增的为了提前存储必要数据，官方的那四个原始参考都是值留下了函数签名，期待我们的具体实现，所以我查阅了网络资料，实现这些函数应该就能带来不错的优化效果。

```
===== 优化后 =====
#ifndef HPCG_OPTIMIZED_DATA_HPP
#define HPCG_OPTIMIZED_DATA_HPP

#include "SparseMatrix.hpp"

struct HPCGOptimizedMatrix {
  local_int_t nrow;
  local_int_t nnz;
  local_int_t *rowptr;
  local_int_t *col;
  double *values;
  local_int_t *diag;
  double *diagValues;
  bool fastPath;
};

inline HPCGOptimizedMatrix *GetOptimizedMatrix(const SparseMatrix &A) {
  return static_cast<HPCGOptimizedMatrix *>(A.optimizationData);
}

#endif
```

首先是 OptimizedData.hpp ，这个是新增的，这个数据结构主要负责保存优化后的矩阵数据。建这个数据结构的目的为了以后可以把矩阵中的有效数据集中放在一起，并记录每一行数据的起止位置，这样计算时可以直接找到需要的内容。就比如说，原来的矩阵像一间没有分类的仓库，每次找东西都要从头翻到尾，优化后像把物品按货架和

编号整理好，需要什么时直接找到对应货架，不用重复翻找。这个优化就是免去在访问不规则的存储位置时的查找过程。

```
===== 函数签名 =====
    37  int OptimizeProblem(SparseMatrix & A, CGData & data, Vector & b, Vector & x, Vector & xexact) {
    38
    39    // This function can be used to completely transform any part of the data structures.
    40    // Right now it does nothing, so compiling with a check for unused variables results in complaints
===== 主体 =====
    15    local_int_t offset = 0;
    16    d->rowptr[0] = 0;
    17    for (local_int_t i = 0; i < d->nrow; ++i) {
    18      const int nnz = static_cast<int>(A.nonzerosInRow[i]);
    19      const local_int_t *cols = A.mtxIndL[i];
    20      const double *vals = A.matrixValues[i];
    21      const double *diagptr = A.matrixDiagonal[i];
    22      d->diag[i] = offset + static_cast<local_int_t>(diagptr - vals);
    23      d->diagValues[i] = *diagptr;
    24      for (int j = 0; j < nnz; ++j) {
    25        d->col[offset + j] = cols[j];
    26        d->values[offset + j] = vals[j];
    27      }
    28      offset += nnz;
    29      d->rowptr[i + 1] = offset;
    30    }
    31    A.optimizationData = d;
    36  int OptimizeProblem(SparseMatrix &A, CGData &, Vector &, Vector &, Vector &) {
    37    BuildOptimizedMatrix(A);
    38    return 0;
```

然后是 OptimizeProblem.cpp，这个也是新增的，它主要负责在程序开始时整理一次矩阵，并提前记录每一行对角元素的位置。原理是把后面经常要用的信息提前准备好，避免计算过程中反复查找。

```
===== 原始参考 =====
    40  int ComputeSPMV( const SparseMatrix & A, Vector & x, Vector & y) {
    41
    42    // This line and the next two lines should be removed and your version of ComputeSPMV should be used.
    43    A.isSpmvOptimized = false;
    44    return ComputeSPMV_ref(A, x, y);
    45  }
===== 优化过后 =====
    30    const double *xv = x.values;
    31    double *yv = y.values;
    32    for (local_int_t i = 0; i < d->nrow; ++i) {
    33      double sum = 0.0;
    34      for (local_int_t k = d->rowptr[i]; k < d->rowptr[i + 1]; ++k)
    35        sum += d->values[k] * xv[d->col[k]];
    36      yv[i] = sum;
    37    }
    38    A.isSpmvOptimized = true;
    39    return 0;
```

这个是 ComputeSPMV.cpp ，它负责完成稀疏矩阵与向量相乘。原理是只计算矩阵中真正存在的非零元素，跳过大量空白位置。因为我们这个是稀疏矩阵，所以有大量零元素，这些乘起来肯定是不必要的。

```
===== 原始参考 =====
    50  int ComputeSYMGS( const SparseMatrix & A, const Vector & r, Vector & x) {
    51
    52    // This line and the next two lines should be removed and your version of ComputeSYMGS should be used.
    53    return ComputeSYMGS_ref(A, r, x);
    54
===== 优化过后 =====
    24    const double *rv = r.values;
    25    double *xv = x.values;
    26    for (local_int_t i = 0; i < d->nrow; ++i) {
    27      double sum = rv[i];
    28      for (local_int_t k = d->rowptr[i]; k < d->rowptr[i + 1]; ++k)
    29        sum -= d->values[k] * xv[d->col[k]];
    30      const double diag = d->diagValues[i];
    31      sum += xv[i] * diag;
    32      xv[i] = sum / diag;
    33    }
    34    for (local_int_t i = d->nrow; i-- > 0;) {
    35      double sum = rv[i];
    36      for (local_int_t k = d->rowptr[i]; k < d->rowptr[i + 1]; ++k)
    37        sum -= d->values[k] * xv[d->col[k]];
    38      const double diag = d->diagValues[i];
    39      sum += xv[i] * diag;
    40      xv[i] = sum / diag;
    41    }
    42    return 0;
```

ComputeSYMGS.cpp 是负责完成对称高斯赛德尔计算。原理是利用已经整理好的矩阵数据，并直接读取提前保存的对角元素，快速计算每一行的新结果。这个和第二个优化差不多的概念，也是预先准备而不是现算现找。

```
===== 原始参考 =====
    33  int ComputeMG(const SparseMatrix  & A, const Vector & r, Vector & x) {
    34
    35    // This line and the next two lines should be removed and your version of ComputeSYMGS should be used.
    36    A.isMgOptimized = false;
    37    return ComputeMG_ref(A, r, x);
    38  }
===== 优化过后 =====
     7  int ComputeMG(const SparseMatrix &A, const Vector &r, Vector &x) {
     8    ZeroVector(x);
     9    int ierr = 0;
    10    if (A.mgData != 0) {
    11      for (int i = 0; i < A.mgData->numberOfPresmootherSteps; ++i)
    12        ierr += ComputeSYMGS(A, r, x);
    13      if (ierr) return ierr;
    14      ierr = ComputeSPMV(A, x, *A.mgData->Axf);
    15      if (ierr) return ierr;
    16      ierr = ComputeRestriction_ref(A, r);
    17      if (ierr) return ierr;
    18      ierr = ComputeMG(*A.Ac, *A.mgData->rc, *A.mgData->xc);
    19      if (ierr) return ierr;
    20      ierr = ComputeProlongation_ref(A, x);
    21      if (ierr) return ierr;
    22      for (int i = 0; i < A.mgData->numberOfPostsmootherSteps; ++i)
    23        ierr += ComputeSYMGS(A, r, x);
    24    } else {
    25      ierr = ComputeSYMGS(A, r, x);
    26    }
    27    A.isMgOptimized = true;
    28    return ierr;
```

ComputeMG.cpp 负责连接多重网格中的各个计算步骤。原理是让多重网格在不同层次上进行计算时，都使用前面已经优化过的矩阵乘法和对称高斯赛德尔计算。这里可以从官方的注释就可以看出来

```
===== 原始参考 =====
 42  int ComputeWAXPBY(const local_int_t n, const double alpha, const Vector & x,
 43      const double beta, const Vector & y, Vector & w, bool & isOptimized) {
 44
 45    // This line and the next two lines should be removed and your version of ComputeWAXPBY should be used.
 46    isOptimized = false;
 47    return ComputeWAXPBY_ref(n, alpha, x, beta, y, w);
 48  }
===== 优化过后 =====
  3  int ComputeWAXPBY(const local_int_t n, const double alpha, const Vector &x,
  4                    const double beta, const Vector &y, Vector &w,
  5                    bool &isOptimized) {
  6    const double *xv = x.values;
  7    const double *yv = y.values;
  8    double *wv = w.values;
  9    if (alpha == 1.0) {
 10      for (local_int_t i = 0; i < n; ++i) wv[i] = xv[i] + beta * yv[i];
 11    } else if (beta == 1.0) {
 12      for (local_int_t i = 0; i < n; ++i) wv[i] = alpha * xv[i] + yv[i];
 13    } else {
 14      for (local_int_t i = 0; i < n; ++i) wv[i] = alpha * xv[i] + beta * yv[i];
 15    }
 16    isOptimized = true;
 17    return 0;
 18  }
```

ComputeWAXPBY.cpp 是负责完成两个向量的加权相加。原理是直接在一次循环中计算每个位置的结果，并对系数为一的常见情况进行简化。打个比方，给两箱货物按比例混合时，原来可能要先分别称重、记录、再计算，优化后可以按照同一个流程直接完成；如果两箱货物都不需要调整比例，就直接相加即可。

```
===== 原始参考 =====
 41  int ComputeDotProduct(const local_int_t n, const Vector & x, const Vector & y,
 42      double & result, double & time_allreduce, bool & isOptimized) {
 43
 44    // This line and the next two lines should be removed and your version of ComputeDotProduct should be used.
 45    isOptimized = false;
 46    return ComputeDotProduct_ref(n, x, y, result, time_allreduce);
 47  }
===== 优化过后 =====
  7  int ComputeDotProduct(const local_int_t n, const Vector &x, const Vector &y,
  8                        double &result, double &time_allreduce,
  9                        bool &isOptimized) {
 10    const double *xv = x.values;
 11    const double *yv = y.values;
 12    double local = 0.0;
 13    if (xv == yv) {
 14      for (local_int_t i = 0; i < n; ++i) local += xv[i] * xv[i];
 15    } else {
 16      for (local_int_t i = 0; i < n; ++i) local += xv[i] * yv[i];
 17    }
 18  #ifndef HPCG_NO_MPI
 19    const double t0 = mytimer();
 20    MPI_Allreduce(&local, &result, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
 21    time_allreduce += mytimer() - t0;
 22  #else
 23    result = local;
 24  #endif
 25    isOptimized = true;
 26    return 0;
```

最后是 ComputeDotProduct.cpp ，他负责计算两个向量对应元素相乘后的总和。原理是先让每个进程完成自己负责部分的累加，最后再把各个进程的结果汇总起来。

函数优化的部分就是这些了，优化完成之后，让我们跑测试看一

看。

```
HPCG result is VALID with a GFLOP/s rating of: 3.47223
Results are valid but execution time (sec) is: 31.1962
HPCG result is VALID with a GFLOP/s rating of: 3.49295
Results are valid but execution time (sec) is: 31.05
HPCG result is VALID with a GFLOP/s rating of: 3.46149
Results are valid but execution time (sec) is: 31.3385
HPCG result is VALID with a GFLOP/s rating of: 3.46703
Results are valid but execution time (sec) is: 30.0141
HPCG result is VALID with a GFLOP/s rating of: 3.49639
Results are valid but execution time (sec) is: 31.016
```

可以看到，这五次测试依旧是在 nx=64、ny=64、nz=64 的问题规模下进行的，将原来 1.7~1.8 左右的 GFLOP/s 优化到 3.5 GFLOPS/s 左右，对于优化的稳定性，我想将问题规模进一步扩大，看看数据规模增大之后还能不能有明显的优化效果。

```
HPCG result is VALID with a GFLOP/s rating of: 3.21162
Results are valid but execution time (sec) is: 33.9707
HPCG result is VALID with a GFLOP/s rating of: 3.26551
Results are valid but execution time (sec) is: 33.6268
HPCG result is VALID with a GFLOP/s rating of: 3.25617
Results are valid but execution time (sec) is: 33.6736
HPCG result is VALID with a GFLOP/s rating of: 3.2352
Results are valid but execution time (sec) is: 33.9345
HPCG result is VALID with a GFLOP/s rating of: 3.25791
Results are valid but execution time (sec) is: 33.7057
```

这个是将问题规模扩大到 nx=128、ny=128、nz=128 之后的测试结果，可以看到 GFLOP/s 略有降低，但是总体优化性能还是能维持在 3 GFLOP/s 以上，优化效果比较稳定。

## 四、总结

在本次任务中，我顺利安装并成功运行了 HPL 和 HPCG 的基准测试，并通过调整 HPL 参数配置和优化 HPCG 核心函数，成功使得 HPL 从基准性能 62.186 GFLOP/s 提升到 174.5 GFLOP/s、HPCG 从基准性能 1.83 GFLOP/s 提升到 3.47 GFLOP/s。